# 4. Proiectarea arhitecturală şi implementarea programelor. Implementarea scenariilor pentru formularul de acizitie a materialelor

Până aici ne-am ocupat de:

- ✓ Elaborarea structurii aplicatiei, redată sub forma a) claselor de obiecte și a relațiilor dintre ele, b) tabelelor bazei de date și a legăturilor dintre ele și c) maparea dintre cele 2 straturi. Structura aplicației s-a concretizat prin construirea modelului fizic (vezi partea 3) pe baza transformării modelelor conceptual și logic.
- ✓ Completarea specificațiilor de proiectare privind comportamentul aplicației, ca rezultat al proiectării formularelor și rapoartelor. Desigur că este posibil ca o parte din comportamentul aplicației să nu se regăsească în formularele și rapoartele aplicației, iar astfel de prelucrări vor putea fi inițiate de utilizatori direct din meniul aplicației. Astfel de prelucrări au fost ignorate in acest model deoarece dpdv al proiectării aplicației nu implică aspecte specifice.

În continuare ne vom preocupa de implementarea comportamentului aplicației ținând cont de modelul fizic obținut anterior.

Asa cum ne-am obisnuit deja, dupa descrierea narativa a cerintelor unui sistem (fie de structură – *diagrama de clase, fie de comportament – formularul descris cap. 2) va urma un proces de traducere mai întâi* intr-un model logic si apoi in modelul fizic.

Implementarea comportamentului unei aplicatii trebuie sa aiba INTOTDEAUNA la baza o arhitectura si unul sau mai multe sabloane de proiectare (design patterns) bine definite. Cu alte cuvinte, trebuie sa existe o anumita consistenta intre secventele finale de cod care implementeaza doua sau mai multe scenarii (e.g. formulare) diferite. Altfel spus, este vorba despre normalizarea codului pe baza unor reguli clare ce vor permite: reutilizarea codului, evitarea redundantei, managementul componentelor si al cuplarii intre acestea. Da, suna cunoscut: este un proces similar celui de normalizare a datelor intr-o baza de date. Scopul final este:

**1) de a scrie o secventa de cod o singura data;**
**2) de a o reutiliza ori de cate ori este posibil pentru a rezolva scenarii mai complexe;**
**3) de a obtine programe de calitate, adica usor de intretinut si testat.**

Exista o multitudine de asemenea sabloane aritecturale, fiecare fiind potrivit tipului de aplicatie ce va fi implementat. Astfel, prima intrebare este intotdeauna ce fel de aplicatie dorim sa implementam: desktop, client-server cu doua straturi (client-desktop si server de baze de date), client-server pe trei straturi (client, server aplicatie, server baza de date), aplicatie web sau mobila etc.

In demersul nostru vom selecta un sablon arhitectural si două șabloane de proiectare. Ele pot fi aplicate cu succes in majoritatea situatiilor si folosind orice limbaj. Astfel, pe langa cadrul de lucru ORM deja discutat, in dezvoltarea acestei aplicatii vom mai utiliza:

1) *MVC* (Model-View-Controller) – un sablon arhitectural pentru organziarea codului privind interfata utilizator;
2) *Repository*- un sablon de proiectare specific aplicatiilor de gestiune date (enteprise applications);
3) *Adapter* – un sablon general valabil care poate fi aplicat in orice aplicatie software.

## 4. 1 Proiectarea şi implementarea componentei pentru gestiunea accesului la stratul de persistenta (date)

Revenim la modelul aplicatiei. In partea a 3-a am aratat mecanismele de traducere a regulilor economice structurale ce descriu scenariile de analiza. Este momentul sa adaugam si partea de comportament. *Aceasta va evolua progresiv* in functie de necesitatile clientilor (formularele/ecranele in cazul nostru).
Aplicatia noastra este orientata spre managementul datelor, prin urmare aspectele privind accesul la date sunt importante. Astfel, in modelul nostru, componentele necesar al fi dezvoltate se rezuma, deocamdata, la operatii **CRUD** (Create, Read, Update, Delete) ce vor fi utilizate de diverse formulare si rapoarte. Intrebarea este cum organizam operatiile respective pe clase si componente. In acest sens, referitor la operatii CRUD, exista doua sabloane bine cunoscute: DAO (Data Access Objects) si Repository. Pe scurt:

- Ambele sabloane au un obiectiv comun clar definit: *ascunderea implementarii (adica a tehnologiilor) privind accesul la baza de date* (utilizare directa JDBC sau folosirea unui framework ORM precum JPA sau JDO etc) si expunerea catre clienti a unor metode care utlizeaza exclusiv obiecte din modelul fizic de structura. Cu alte cuvinte, aplicatiile-client (formularele, in cazul nostru) nu trebuie sa fie „constiente" despre mecanismul utilizat pentru accesul la baza de date. Asta inseamna ca nu trebuie sa utilizeze in mod direct nici componente specifice JPA (spre exemplu, EntityManager) nici componente de baza JDBC. Pentru operatii CRUD, clientul va folosi in mod exclusiv API-ul specific aplicatiei, pus la dispozitie sub forma de componente DAO sau Repository.
- Diferenta majora intre cele doua este legata de faptul ca se creeaza si se asociaza cate un DAO fiecarei entitati in timp ce Repository are in vedere clasele radacina dintr-un model si este considerata un tip mai special de colectie ce utilizeaza baza de date ca spatiu de stocare. In mod concret, pentru modelul nostru ar trebui sa dezvoltam 10 DAOs (sunt 10 clase-entitate), in timp ce numarul de Repositories ramane la latitudinea fiecarui proiectant.Pentru modelul nostru o varianta intuitiva ar fi: un repository pentru tot ce tine de documente (Document e clasa radacina pentru receptii, transferuri si consumuri) si un altul pentru asa numitele entitati-satelit (de sine statatoare, ce nu depind de radacina dar de care radacina depinde - intr-o exprimare clasica din domeniul BD: nomenclatoare).

Astfel, vom crea doua asemenea clase/componente: DocumentRepository si MasterRepository (vezi figura 11).

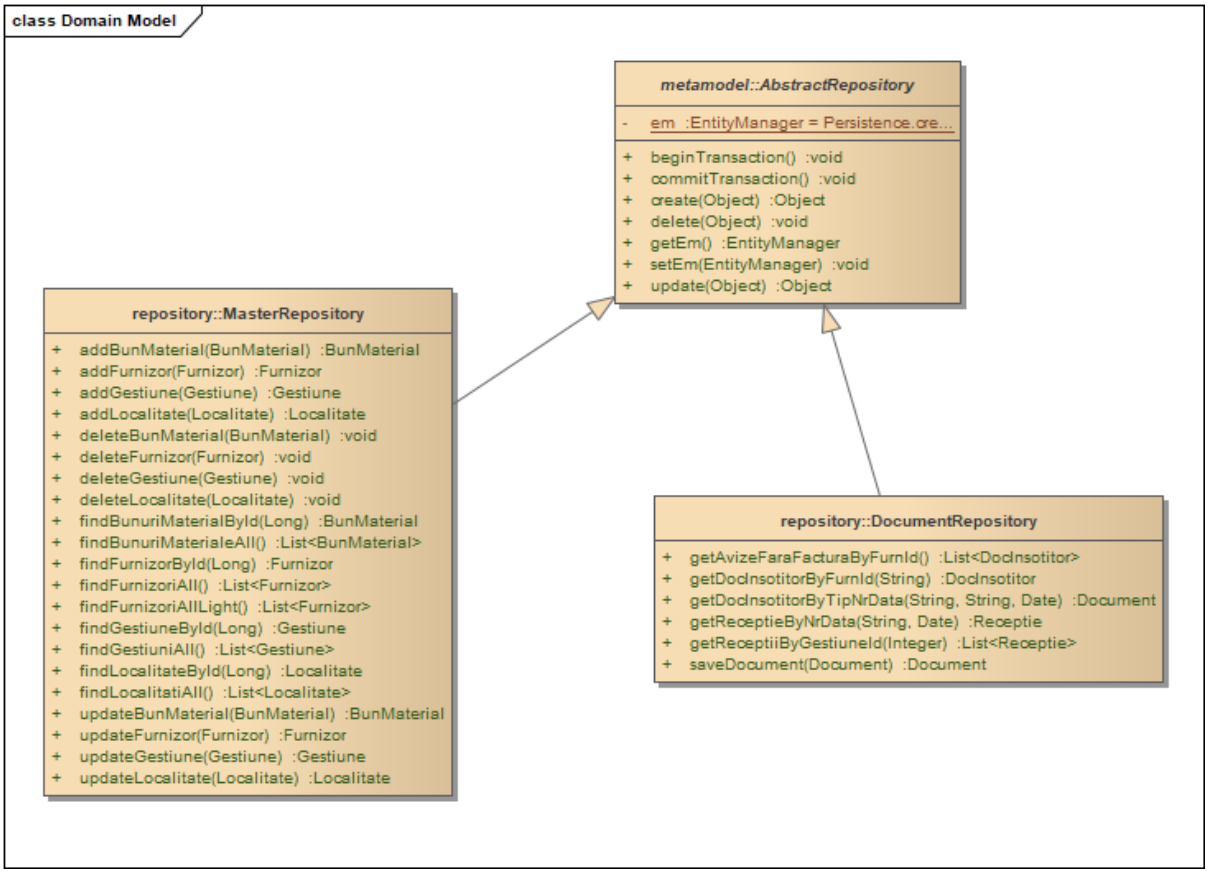


**Figura 11 Clasele/Componentele DocumentRepository si MasterRepository**

Primul si cel mai important aspect pe care trebuie sa-l observam in modelul din figura 11 este legat de ascunderea implementarii accesului la baza de date (asa cum spuneam mai sus, unul din obiectivele principale). Clasa AbstractRepository incapsuleaza intr-o maniera inalt abstractizata operatiile fundamentale CRUD folosind componentele JPA. Observati ca metodele primesc drept argumente tipul Object si returneaza acelasi tip Object. Dupa cum stim, in Java, orice clasa extinde in mod implicit clasa Object si, ca urmare, orice obiect este de tip Object. Asadar, metodele declarate in clasa AbstractRepository vor fi capabile sa comunice cu baza de date pentru orice entitate primita drept argument. Acest lucru se realizeaza prin utilizarea interna a componentei EntityManager, ce furnizeaza, la randul ei, un API inalt abstractizat.
„Ascunderea implementarii" presupune ca un client sa nu poata utiliza in mod direct aceasta clasa. Acest deziderat se implementeaza prin declararea clasei ca fiind de tip *abstract*.

Ulterior, implementarea claselor/componentelor DocumentRepository si MasterRepository extind si utilizeaza aceasta clasa oferind mai departe clientilor metode CRUD specifice aplicatiei, bine definite si non-ambigue. Spre exemplu:implementarea metodei DocumentRepository.saveDocument (Document d):

```
public class DocumentRepository extends AbstractRepository {

	@Override
	public Document saveDocument(Document document) {
		if (document.getId()==null)//obiectnou
			document=(Document)this.create(document);
		else//obiect existent in BD
			document=(Document)this.update(document);

		return document;
	}
```

unde metodele *create* si *update* arata astfel, in super-clasa AbstractRepository:

```
privatestatic EntityManager em = Persistence.createEntityManagerFactory(
                  "SGSMPersistenceUnit").createEntityManager();

      public Object create(Object o) {
            this.getEm().persist(o);
            return o;
      }

      public Object update(Object o) {
            Object managedEntity = this.em.merge(o);
            return managedEntity;
      }
```

Codul de mai sus ar trebui sa va fie familiar de la disciplina Programare 2.
*Atentie! Intotdeauna metodele de tip Create si Update (vezi si cele din clasele-repository) vor returna o noua versiune a obiectului, dupa ce se executa persistenta in BD*. Acest aspect este esential pentru managementul corect al actualizarilor concurente si implementarea mecanismului OptimisticLocking.

Iata in continuare, pas-cu-pas, procesul de implementare si deciziile-cheie luate (**vezi si figura 11 precum si implementarea SGSM-Model**):

1) Definim:
   a. MasterRepository – operatii CRUD pentru Localitate, Furnizor, Gestiune, BunMaterial
   b. DocumentRepository – operatii CRUD pentru documente – metodele aici se adauga pe masura ce sunt necesare. Deocamdata, pentru primul test al viitorului formular avem nevoie de o metoda saveDocument (va fi executata de butonul Save din formular) care ofera suport pentru persistenta oricarui obiect de tip Document (deci si subclasele acestuia). Aceasta metoda va efectua o verificare interna pentru a decide operatia CREATE sau UPDATE ce trebuie efectuata.

2) Se alege strategia de implementare a claselor Repository privind accesul la baza de date. Am discutat mai sus aceste aspecte. De retinut ca trebuie sa ne pastram libertatea de a modifica ulterior implementarea fara a afecta clientii (incapsulare / ascunderea implementarii).

3) Am spus deja ca am ales o strategie ORM sub forma specificatiei JPA cu implementarea Hibernate. Asta presupune ca fiecare operatie SQL nu va fi gestionata direct de catre programator ci prin intermediul componentei special desemnate, EntityManager (vezi curs Programare II) care lucreaza cu un limbaj similar SQL, denumit JPA-QL. Asadar, fiecare Repository va trebui sa obtina un obiect EntityManager configurat pe baza descrierii din fisierul META-INF/persistence.xml. Cum obtinem o referinta catre aceasta componenta si unde tinem instanta respectiva? Membrul static AbstractRepository.em folosit de metodele generalizate CRUD din aceeasi clasa este raspunsul. De ce un atribut static? Pentru a obtine un raspuns, cititi mai departe.

4) Un aspect esential in aplicatiile de gestiune este dat de managementul tranzactiilor (vezi acronimul ACID si semnificatia acestuia). Sunt cateva principii in implementarea managementului tranzactiilor:

   a. **Tranzactia trebuie intotdeauna gestionata (initiata, comisa, anulata) de catre Client**! In cazul nostru, primul client al acestor componente va fi chiar formularul de achizitii materiale. Asadar, trebuie oferita clientului o modalitate de a initia o tranzactie si de a comite tranzactia respectiva. In mod normal, este necesara si o operatie de anulare a tranzactiei dar, asa cum spuneam, KISS ne invata ca pana nu avem nevoie efectiv de aceasta operatie, nu o vom adauga. In acest moment nu avem nevoie de o asemenea operatie deoarece fiecare repository va folosi, prin intermediul clasei AbstractRepository

componenta JPA EntityManager care anuleaza automat intreaga tranzactie atunci cand intervine o eroare in operatiile SQL cu baza de date. Intrebarea este cum oferim clientilor mecanismul de management al tranzactiei stiind ca acestia nu vor avea acces direct la obiectul EntityManager (in principiu, nici nu vor sti ca utilizam JPA, daca nu le oferim o documentatie a implementarii). Raspuns: **Super**-clasa AbstractRepository oferă implementarea metodei beginTransaction(). Iata o clasa de test – parte integranta a proiectului SGSM-Model – care constituie un prim client (testele sunt intotdeauna primii clienti ai aplicatiei):

```
publicclass TestLocalitati {

      static MasterRepository repo = new MasterRepository();

      public static void main(String[] args) {
            List<Localitate> localitati = repo.findLocalitatiAll();
            //dacanusuntlocalitatiintroducemcateva date de test
            if (localitati.size() == 0) {
                  addLocalitati();
                  localitati = repo.findLocalitatiAll();
            }
            //verificamdacasunt date de test (artrebuisaexiste, laacest moment)
            Assert.assertTrue(localitati.size()>0);
      }

      Private static void addLocalitati() {
            repo.beginTransaction();
            for (int i = 1; i < 10; i++) {
                  Localitate loc = new Localitate();
                  loc.setCod(100 + i);
                  loc.setDenumire("Localitate " + (100 + i));

                  repo.addLocalitate(loc);
            }
            repo.commitTransaction();
      }
```

*Observati ca tranzactia este initiata numai pentru operatii de persistenta (pentru optimizare maxima, este recomandat a nu include interogarile in tranzactii decat in situatii bine definite care necesita garantarea consistentei datelor citite de mai multe ori). Observati de asemenea ca tranzactia este initiata in afara buclei for, ceea ce inseamna ca vom avea fie toate cele 10 localitati noi inserate in BD, fie nici una (in caz de eroare la macar una dintre operatiile de creare Localitate).*

b. **Pentru a evita conflictele posibile intre tranzactii initiate simultan (sau aproape) de mai multi utilizatori trebuie sa existe o singura conexiune activa care sa gestioneze toate tranzactiile la nivelul aplicatiei.** In acest sens, exista cadre de lucru bine definite pentru aplicatiile de tip client-server. In exemplul nostru, insa, aplicam cu tenacitate principiul KISS (Keep-It-Short-and-Simple). Cat timp aplicatia ramane simpla, utilizabila local sau pe un server Web, putem apela la un sablon de proiectare clasic: **singleton** – presupune ca un singur obiect (instanta) dintr-o anumita clasa sa fie utilizat de catre intreaga aplicatie (aici o implementare rudimentara pentru claritate). In exemplul nostru, obiectul care gestioneaza efectiv tranzactia este o instanta EntityManager (binecunoscuta componenta JPA). Pentru ca toata aplicatia sa utilizeze un singur obiect EntityManager, acesta trebuie declarat ca un membru *static*. Pentru ca toate obiectele Repository trebuie sa obtina acelasi obiect EntityManager:

   i. a fost definita super-clasa *AbstractRepository*
   ii. A fost definit si instantiat atributul *static* em de tip EntityManager
   iii. Toate clasele repository extind aceasta clasa si, la instantierea primului Repository, se va initializa si membrul *em*, dupa care toate instantierile succesive vor folosi exact acelasi obiect.

## 4.2 Testarea componentei de gestiune a datelor

Acum, primii clienti ai unei aplicatii sunt INTOTDEAUNA testele unitatilor de cod (unit testing) – adica acele secvente de cod care verifica fiecare metoda ce constituie API-ul expus public. Sunt deja exemplificate in proiectul SGSM-model, folderul *test*, o mare parte a unor asemenea teste. In principiu, avem nevoie oricum de date de test asa incat, in loc de a le introduce cu fraze SQL direct in BD, e mai indicat a scrie cateva teste care sa verifice API-ul si sa adauge si date de test (2in1, ca-n reclame...)
Am vazut deja un asemenea test mai sus (TestLocalitati). In cadrul acestor teste veti observa utilizarea in mod repetat a clasei Assert. Aceasta clasa face parte din framework-ul JUnit – unul din cele mai utilizate (daca nu chiar universal utilizat) in scrierea testelor pentru aplicatii Java. ClasaAssert ofera o suita de metode foarte convenabile prin intermediul carora programatorii pot verifica daca rezultatele intermdiare/finale sunt si cele asteptate fara a utiliza structuri de tip if. Astfel:

```
Assert.assertTrue(localitati.size()>0);
```

va genera o eroare daca dimensiunea colectiei localitati este zero
Iar

```
Assert.assertTrue(„Nu exista localitati! Adaugati cateva”,
localitati.size()>0);
```

va genera aceeasi eroare doar ca mesajul va fi cel precizat de primul parametru

## 4.3 Proiectarea şi implementarea componentei ce gestionează comportamentul specific formularului

În figura 12 este prezentat şablonul arhitectural Model-View-Controller. El este format din:

Model

- Furnizeaza datele pentru stratul de prezentare
- De regula:
  - se construieste cate un model de date pentru fiecare formular (forma grafica)
  - acest model nu aduce nimic nou fata de modelul domeniului (modelul structural prezentat in partea I si modelul comportamental ce urmeaza a fi discutat) ; doar ***adapteaza*** structurile din modelul domeniului la necesitatile interfetei grafice

Controller

- traduce evenimentele generate de interfata grafica in apeluri catre metodele modelului (delegare)
- executa logica specifica pentru controlul interfetei grafice (mesaje de eroare, activare/dezactivare obiecte grafice etc)

View

- executa logica de prezentare (afisarea obiectelor grafice)
- conecteaza obiectele grafice la datele modelului (citeste modelul)
- actualizeaza modelul structural cu datele din componentele grafice (modifica modelul)
- trimite evenimente controller-ului

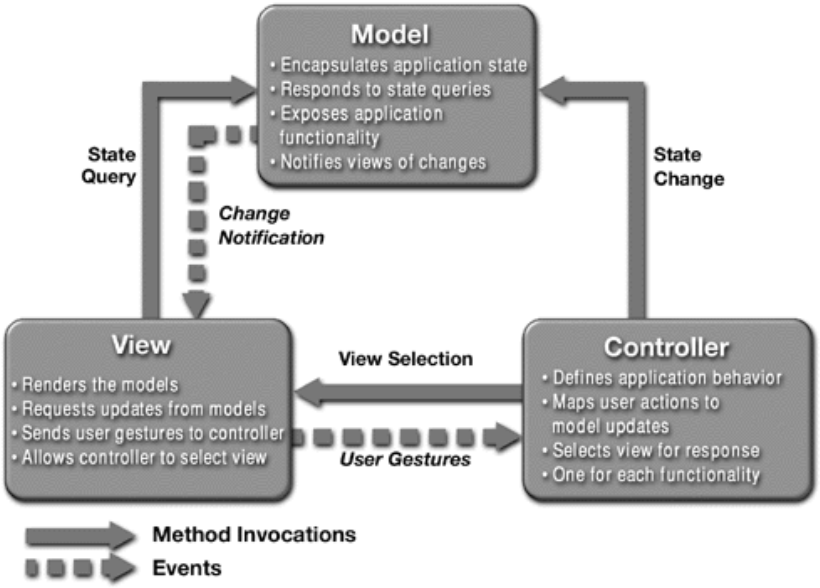


Figura 12 Sablonul architectural Model-View-Controller

sursa : http://java.sun.com/blueprints/guidelines/designing_enterprise_applications_2e/app-arch/app-arch2.html

In aplicarea șablonului arhitectural MVC, noi vom introduce o a 4-a componenta, care are rolul de a adapta componenta Model în funcție de nevoile de date din formular. Această componentă este rezultatul aplicării șablonului de proiectare Adapter.

**Metoda de lucru pentru implementarea comportamentului specific formularelor folosind șablonul MVC:**

Pentru fiecare formular proiectat se vor realiza trei activitati :

1. Se adauga câte o clasa cu rol de ModelAdapter si o alta cu rol de Controller. Modelul formularului este intotdeauna o adaptare a modelului conceptual (modelul domeniului).
2. ITERATIE: Pentru fiecare zona din formular se implementeaza sursa de date in clasa-ModelAdapter.
3. ITERATIE: Pentru fiecare eveniment-tinta (butoane, selectii etc) din formular se adauga în clasa cu rol de Controller o metoda de interceptare cu scop de a implementa logica de executie.

### 4.3.1 Implementarea formularului de achizitie a materialelor

*Nota*. Pentru implementarea formularelor s-a creat un proiect nou: SGSM-GUI (vezi arhiva de pe Portal FEAA). Acesta constituie prima aplicatie-client pentru modelul nostru. Acest proiect este dependent de SGSM-Model si acest aspect trebuie declarat explicit: vezi Project/Properties/Build Path.

Implementare pas-cu-pas.

1. **Declarare ModelAdapter si Controller**. In proiectul SGSM-Model adaugam o clasa cu rol de Controller - *AchizitiiFormCtrl* – ce ***va avea o relatie de compunere*** cu o noua clasa cu rol de adaptare pentru Model in functie de nevoile de date din formularul de achizitii – *AchizitiiFormData*. *AchizitiiFormCtrl* va defini si implementa toate metodele pentru tratarea evenimentelor de interes iar *AchizitiiFormData* va pastra datele pentru instanta curenta a formularului (documentul current vizibil pe ecran, datele pentru fiecare lista derulanta, tabele etc). Figura 13 arata varianta finala a acestor doua clase, variantă ce va fi obtinuta dupa parcurgerea tuturor pasilor descrisi.

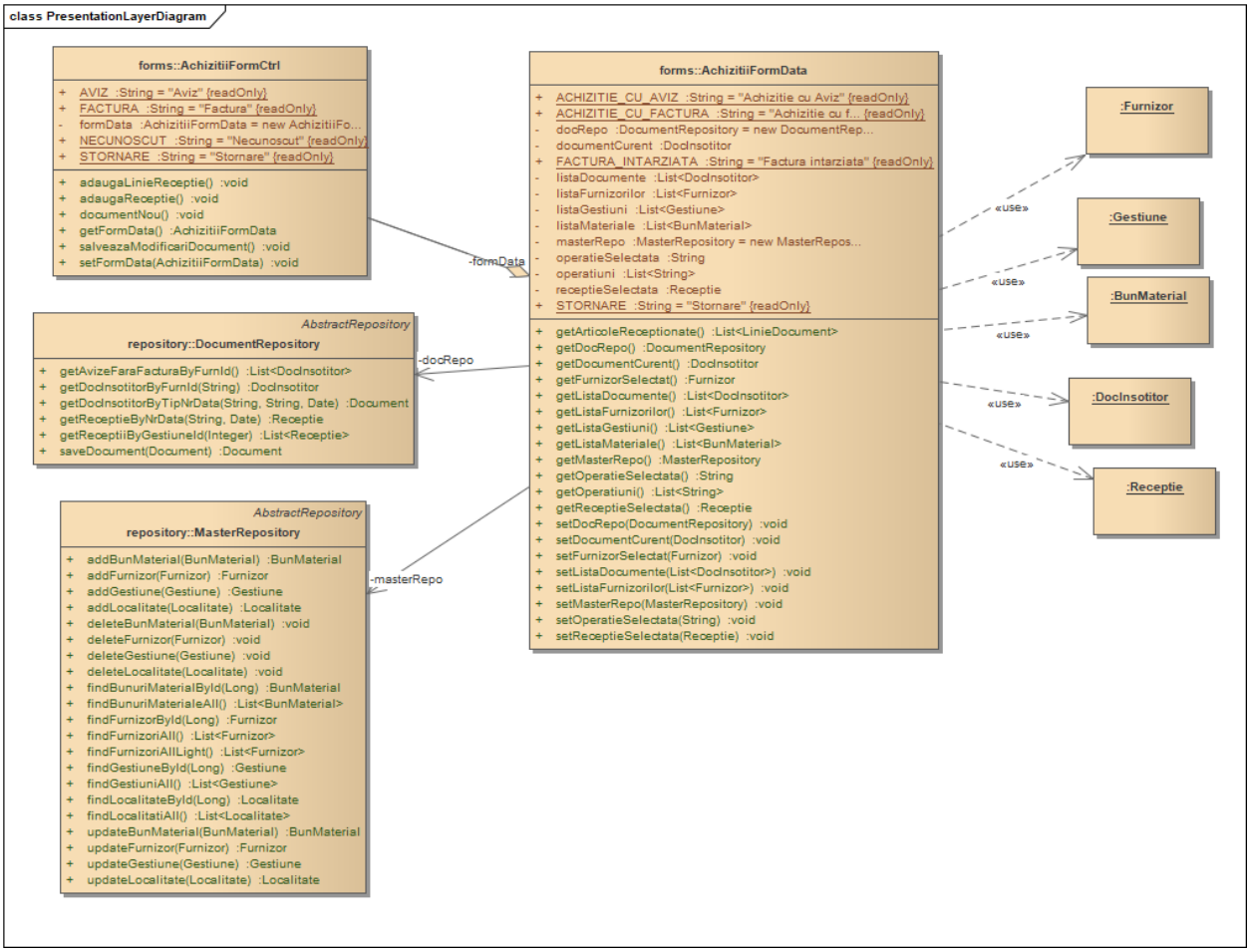


Figura 13 – Clasele utilizate pentru implementarea comportamentului formularului pentru achizitii de materiale

2. **Implementarea surselor de date in clasa ModelAdapter**. Aceasta etapa presupune adaugarea, in clasa AchizitiiFormData a tuturor membrilor necesari pentru sustinerea datelor afisate de obiectele grafice ale formularului. Dupa cum spuneam, modelul de date al formului (AchizitiiFormData) are rolul de a prezenta o adaptare a modelului domeniului, adaptare necesara pentru re-organizarea si filtrarea datelor in functie de necesitatile formularului. Este vorba de doua tipuri de obiecte ce vor fi tinute de acest adapter:
    a. *Entitati pentru stocare date* – noi vom utiliza in mod direct clasele din model dar, pentru anumite scenarii, exista si varianta de aborare DTO (Data Transfer Objects – vezi pe wikipedia).
    b. *Selectie clase Repository pentru operatii de persistenta*. Sunt necesare operatii de persistenta cu urmatoarele tipuri de obiecte: DocInsotitor, Receptie, BunMaterial, Furnizor, Gestiune. Ca urmare, vor fi utilizate ambele clase cu rol de Repository: MasterRepository, DocumentRepository.

Metoda de lucru e foarte intuitivă: pentru fiecare zona din formular trebuie sa avem o zona de date bine definite. Nu vom prezenta aici implementarea pentru toate zonele, din moment ce exista comentarii detaliate in clasa AchizitiiFormData din proiectul SGSM-GUI. Doar cateva reguli pe tipuri de scenarii:

- Exista o zona comuna tuturor formularelor (zona 0) ce se refera la obiectul ale carui proprietati sunt afisate pe ecran precum si la obiectele specializate in acces la date (Repository, in cazul nostru). Adica, orice formular are un obiect-tinta, iar in cazul nostru acest obiect va fi de tip DocumentInsotitor. Iata de ce, vom avea sigur :
    a. o proprietate care sa tina obiectul afisat la un moment dat pe ecran si
    b. o colectie de obiecte obtinute un urma unui process de cautare (optional)
    c. cate o referinta catre fiecare obiect de tip Repository necesar pentru acces la date

```
// --------------ZONA 0 - COMUNA TUTUROR FORMULARELOR -------------------//

      // obiectul-tinta (sursade date) alformularului - celaflatcurent in
      // editare
      // (in cazulnostrupoatefifacturasauaviz)

      private DocInsotitor documentCurent;

      // colectiadeobiecte-tintaobtinute in urmauneioperatiidecautare
      private List<DocInsotitor>listaDocumente;

      // obiectede tip Repository necesarepentruinterogareamodelului
      Private MasterRepository masterRepo = new MasterRepository ();
      Private DocumentRepository docRepo = new DocumentRepository ();
```

- La implementarea sursei de date **pentru liste de tip drop-down trebuie avute in vedere**:
  a. Optimizarea maxima a accesului la date:
     i. de regula se lucreaza cu asa-numite "cache"-uri (este si cazul de fata, vezi codul de mai jos) – modelul formularului pastreaza local o colectie de date si acceseaza baza de date numai in doua situatii:
        1. la prima invocare/initializare;
        2. la o comanda explicita din partea utilizatorului (numai daca se doreste ca formularul sa ofere asa ceva)
  b. Optimizarea memoriei RAM – tine de dimensiunea obiectelor si numarul lor si impune obligatoriu optimizare. In acest sens, in cazul de fata a fost adaugata metoda MasterRepository.`findFurnizoriAllLight` care extrage doar informatiile strict necesare din BD (vezi, in cod, comentariile asociate respectivei metode.

```
// ---------------ZONA 1 - FURNIZORI --------------//
      /**
       * Pentruincarcareaelementelor din lista (2) cboTipOperatie. Obiectulde
       * sincronizare selectie corespunzator este documentCurent.furnizor
       */
      private List<Furnizor>listaFurnizorilor;

      /**
       * implementare standard pentru o colectie-sursa de date a unui obiect de
       * tip lista
       *
       * @return
       */
      public List<Furnizor> getListaFurnizorilor() {
            if (this.listaFurnizorilor == null) // e primul apel (userul tocmai a
            // deschis formul)
            {
                  // this.masterRepo.findFurnizoriAll();//varianta neoptimizata va
            // dura mai mult si va returna si obiecte Localitate, atasate prin
            // relatia Furnizor-ManyToOne-Localitate. In cazul nostru nu avem
            // nevoie de obiecte Localitate atasate obiectelor Furnizor
            this.listaFurnizorilor = this.masterRepo.findFurnizoriAllLight();
            }
            returnlistaFurnizorilor;
      }

      publicvoid setListaFurnizorilor(List<Furnizor> listaFurnizorilor) {
            this.listaFurnizorilor = listaFurnizorilor;
      }
```

c. Obiectul de sincronizare al selectie curente (atat pentru afisare cat si in urma selectiei unui element de catre utilizator) – in 99,9% din cazuri este o relatie Many-To-One a obiectului-tinta al formularului. Iata si implementaarea noastra:

```
// Getteri si Setteri pentru manevrarea slectiei curente.
// lucreaza cu obiectul atasat documentului aflat in editare.
public Furnizor getFurnizorSelectat() {
      returnthis.documentCurent.getFurnizor();
}

/**
 * ATENTIE - in majoritatea mediilor de dezvoltare, obiectele grafice de tip
 * lista returneaza un id (o valoare atomica / un singur atribut) al
 * obiectului selectat din lista, sau, precum in cazul nostru, un obiect
„ligth” (incomplet, minumum de date pentru optimizare memorie RAM). Ca urmare,
trebuie incarcat din BD obiectul corespunzator pentru relatia ManyToOne mai ales
ca, in cazul formularului nostru - vezi scenariul -, multe informatii sunt afisate
despre furnizorul selectat in zona grafica (1)
 *
 * @param furnizorSelectat
 */
publicvoid setFurnizorSelectat(Furnizor furnizorSelectat) {
      Furnizor furnizorComplet = this.masterRepo
                  .findFurnizorById(furnizorSelectat.getId());
      this.documentCurent.setFurnizor(furnizorComplet);
}
```

- ATENTIE la mecanismul de implementare **a sursei de date pentru tabelul GrdLiniiDocI** (zona 4) – s-a luat decizia adaugarii unei noi metode in clasa DocInsotitor (deci in modelul domeniului si nu in modelul formularului). Intr-adevar, respectam aici principiul OO conform caruia metoda de prelucrare trebuie sa fie cat mai “aproape” de locatia datelor-sursa:

```
/**
 * Furnizeaza o listacumulativacutoatematerialelereceptionate
 *
 * @return-listamaterialelorreceptionate- READ ONLY
 */
public List<LinieDocument>getArticoleReceptionate() {
      List<LinieDocument> artReceptionate = new LinkedList<LinieDocument>();
      for (Receptie receptie : receptii)
            for (LinieDocument row : receptie.getLiniiDocument())
                  artReceptionate.add(row);
      // listatrebuiesafie read-only -->nupoatefiutilizatapentru a
      // adaugaliniinoi
      return Collections.unmodifiableList(artReceptionate);
}
```

Apoi, in modelul formularului (AchizitiiFormData) nu facem decat sa delegam responsabilitatea acestei noi metode pornind de la documentul current in editare:

```
public List<LinieDocument> getArticoleReceptionate() {
      returnthis.documentCurent.getArticoleReceptionate();
}
```

- **Pentru liste de elemente fara suport in baza de date** se utilizeaza constante (de regula de tip Enum, dar simplificat aici la utilizare de constante simple). Exemplu: tipuri de operatiuni. A se utiliza cu maxima precautie - aceasta lista nu va mai fi dinamica si va necesita recompilarea aplicatiei daca trebuie modificata!!!

```
// o suitadeconstantenecesarelistei de tipuri operatiuni
      publicstaticfinal String STORNARE = "Stornare";
      publicstaticfinal String FACTURA_INTARZIATA = "Factura intarziata";
      publicstaticfinal String ACHIZITIE_CU_AVIZ = "Achizitie cu Aviz";
      publicstaticfinal String ACHIZITIE_CU_FACTURA = "Achizitie cu factura";
// adaugam si lista de tipuri de operatiuni:
      private List<String>operatiuni;
```

```
public List<String> getOperatiuni() {
        if (operatiuni == null)// primul acces - userul deschide formul
        {
                operatiuni = new LinkedList<String>();

                operatiuni.add(ACHIZITIE_CU_FACTURA);
                operatiuni.add(ACHIZITIE_CU_AVIZ);
                operatiuni.add(FACTURA_INTARZIATA);
                operatiuni.add(STORNARE);
        }
        returnoperatiuni;
}
```

3. **Implementarea evenimentelor din formular in clasa Controller**. Se asociaza fiecare eveniment ce trebuie tratat cu o metoda in clasa AchizitiiFormCtrl (Controller). Iat cateva exemple de evenimente si metode associate:

- evenimentul click pe butonul *Adaugare* in pagina *Facturi/Avize* – metoda *documentNou();*
- evenimentul click pe butonul *Adaugare* in pagina *Receptii* – metoda *adaugaReceptie();*
- evenimentul click pe butonul *Salvare* – metoda *salveazaModificariDocument()*;
- evenimentul selectia unui furnizor (ce are ca efect incarcarea in formular a datelor furnizorului) – metoda selectieFurnizor().

Figura 14 prezinta un exemplu de descriere a comportamentului aplicatiei (logicii aplicatiei) specific unui eveniment la nivelul formularului cu ajutorul diagramei de secvente. Exemplul se refera la evenimentul click pe butonul *Salvare* - metoda `salveazaModificariDocument()`. De urmarit, in paralel, si implementarea acestei metode in proiectul SGSM-GUI.

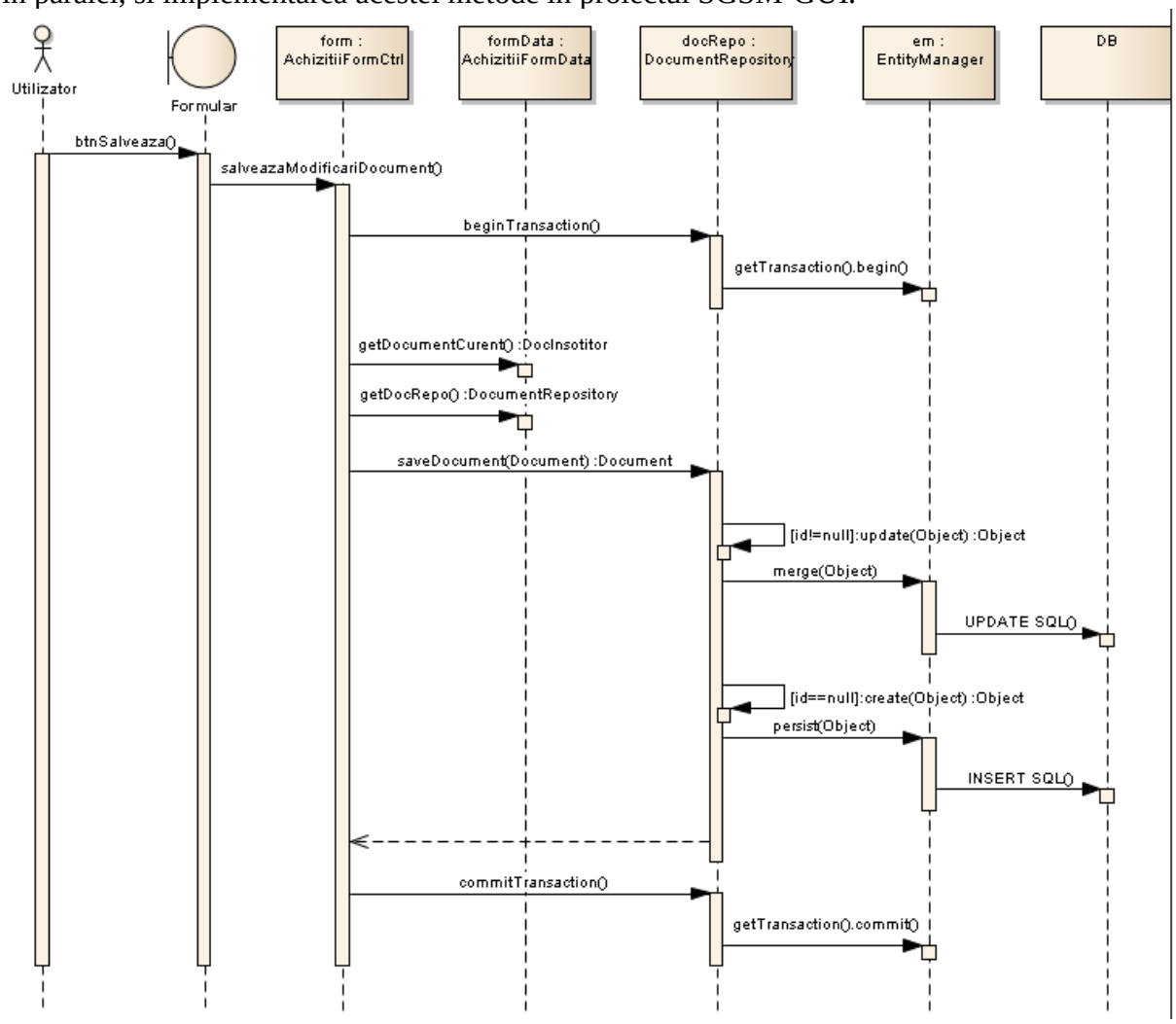


**Figura 14. Diagrama de secvente pentru metoda salveazaModificariDocument()**

In continuare sunt detaliate si interpretate diagrama de secvente din figura 14 si codul implementat:

1) Scenariul incepe prin apasarea de catre utilizator a butonului *Save* in formular, ceea ce are ca efect invocarea metodei *salveazaModificariDocument()* a obiectului form de tipul *AchizitiiFormCtrl* (care joaca rolul de controller).
2) Obiectul form transmite un mesaj obiectului *docRepo* din clasa *DocumentRepository* prin care invoca metoda *beginTransaction()* pentru a se initia tranzactia. Mai departe, obiectul *docRepo* va invoca metodele *getTransaction()* si *begin()* ale obiectului de tip *EntityManager*. Toate metodele din controller, ce au drept scop modificarea stratului de persistenta, vor incepe executia prin crearea unei noi tranzactii. Aceasta va fi inchisa, in final, daca nu apar erori.
3) Obiectul-tinta al acestei metode este chiar obiectul-tinta al intregului formular: un *DocInsotitor* tinut de *FormData.documentCurent*, pe care utilizatorul tocmai l-a modificat folosind interfata grafica si care trebuie trimis catre baza de date. Nu uitam ca *DocInsotitor* este un obiect complex, cu multe relatii OneToMany si ManyToOne, toate aceste relatii fiind configurate pentru persistenta in cascada o data cu obiectul-parinte. In acest sens, obiectul *form* trimite un mesaj obiectului *formData*, instantiat in clasa *AchizitiiFormData*, prin care invoca metoda *getDocumentCurent()*.
4) (optional)La acest moment pot fi adaugate diverse operatii de validare a datelor (daca aceste operatii nu sunt deja introduse la nivelul obiectelor grafice).
5) Mai departe, se invoca metoda `saveDocument()` prin intermediul instantei `DocumentRepository` tinuta de `AchizitiiFormData` (vezi initializarea atributului `docRepo` in AchizitiiFormData) :

```
this.getFormData().getDocRepo().saveDocument(this.getFormData().getDocumentCurent());
```

6) Metoda `saveDocument`, in implementarea `DocumentRepository`, efectueaza o verificare pentru a identifica operatia CRUD necesar a fi executata: Create (Insert) sau Update (update). Se verifica id-ul obiectului. Daca nu exista, inseamna ca e un obiect nou, ce nu exista in BD. In acest caz, obiectul DocRepo isi trimite siesi un mesaj prin care se invoca metoda `create()` mostenita din superclasa `AbstractRepository`. In caz contrar, se executa metoda `update`() mostenita din aceeasi superclasa.

```
public Document saveDocument(Document document) {
		if (document.getId()==null)//obiect nou
			document=(Document)this.create(document);
		else //obiect existent in BD
			document=(Document)this.update(document);

		return document;
	}
```

7) Toate metodele din superclasa `AbstractRepository` lucreaza cu un obiect `EntityManager`, specific API-ului Java-JPA (ORM). Aceasta componenta preia obiectul (oricare ar fi el) si verifica adnotarile JPA (@Entity, @OneToMany etc) asociate clasei din care face parte instanta respectiva. Pe baza acestor adnotari, `EntityManager` genereaza in *mod dinamic instructiuni* SQL Insert/Update/Delete in functie de metoda `EntityManager` apelata (`create/merge/remove`). Instructiunile nu sunt generate numai pentru obiectul primit ci si pentru toate relatiile lui @OneToMany (daca este specificat cascade=CascadeType.ALL) si @ManyToOne. Comenzile SQL se executa efectiv cand tranzactia va fi inchisa.

Asadar, in cazul de fata, se executa una din cele doua metode, *merge()* sau *persist()*, cu un obiect Document

```
public Object create(Object o) {
		this.getEm().persist(o);
		return o;
	}
```

```
public Object update(Object o) {
        Object managedEntity = this.em.merge(o);
        return managedEntity;
}
```

Cum Document este superclasa, orice obiect dintr-o sub-clasa va fi gestionat corect de aceasta metoda, pentru simplul fapt ca, in final, metodele general valabile din EntityManager se executa.

8) Componenta `EntityManager` este initializata in clasa `AbstractRepository` folosind un identificator al unitatii de persistenta definita in META-INF/persistence.xml. In exemplul nostru, "SGSMPersistenceUnit".

```
private static EntityManager em = Persistence.createEntityManagerFactory(
                        "SGSMPersistenceUnit").createEntityManager();
```

Componenta astfel initializata este mostenita de sub-clase. Pentru ca e declarata static, nu numai ca este mostenita dar se va crea o singura instanta tinuta la nivel de clasa si utilizata apoi de toate obiectele din aplicatie prin metoda de acces getEm() asociata. Doar aceasta metoda este declarata publica, fiind astfel posibil ca, pe viitor, sa efectuam schimbari interne privind modul de initialzare si managementul instantelor EntityManager.

9) In final, configurarile asociate unitatii de persistenta `SGSMPersistenceUnit` furnizeaza informatiile privind legatura cu baza de date unde vor fi trimise comezile SQL generate dinamic la pasul 6 (UPDATE sau INSERT).
10) Cand metoda *salveazaModificariDocument*() inchide tranzactia, prin transmiterea mesajului de catre obiectul *DocRepo* prin care se invoca metoda *commitTransaction()*, toate frazele SQL sunt trimise catre BD (vezi consola Eclipse la executia testelor de mai jos). In mod normal, trebuie gestionate eventualele erori, dar nu insistam aici asupra acestui aspect. Important de retinut ca in caz de eroare SQL tranzactia este automat anulata. In cazul altor erori, trebuie anulata explicit (rollback())

### 4.3.2 Testarea evenimentelor de la nivelul formularului

O data implementate metodele (vezi codul sursa si comentariile associate in proiectul SGSM-GUI), inainte chiar de a conecta obiectele grafice, este necesara cel putin o clasa de test care va *simula* actiunile utilizatorilor in formular si va *verifica* rezultatele. In proiectele din lumea reala, **clasele de teste sunt obligatorii** pentru a pastra consistenta si calitatea codului de la o versiune la alta. Aceste teste valideaza noile versiuni din perspectiva functionalitatii déjà implementate (backward compatibility). Sunt si foarte utile in dezvoltarea aplicatiei, sporind mult productivitatea (se executa ca simple POJO), stiut fiind ca testarea reala a interfetelor grafice este, in general, o operatiune costisitoare. Astfel:

1) Simularea presupune executia metodelor ce vor fi asociate cu diferite evenimente, conform pasilor descrisi la definirea scenariului si sarcinilor de lucru ale utilizatorului (vezi partea 2 – Proiectarea interfetei utilizator
2) Verificarea presupune atat validarea starilor prin care trece formularul (AchizitiiFormData) dupa fiecare executie de metoda, cat si succesiunea de stari persistente (in BD) ale modelului de date.

Testul implementat de noi spre exemplificare este TestAchizitiiForm (localizat in folderul *test* al SGSM-GUI) care lucreaza cu asertiuni JUnit (clasa Assert cu suita de metode statice). Pentru simplificare, este un test care poate fi rulat ca aplicatie Java, nu ca un test JUnit.
Din nou, reamintim ca metoda de lucru test-driven-development presupune dezvoltarea graduala a codului de test in paralel cu functionalitatile implementate. Spre exemplu, dupa implementarea zonelor

0 si 1 in AchizitiiFormData, putem deja scrie si executa prima secventa de test din clasa noastra TestAchizitiiForm cu scop de a testa incarcarea corecta a listei de furnizori:

```
AchizitiiFormCtrl form = new AchizitiiFormCtrl();
Assert.assertTrue(
                                "BUG sau Nu exista Furnizori pentru test.
Rulati mai intai testul TestFurnizori din proiectul SGSM-Model",

form.getFormData().getListaFurnizorilor().size() > 0);
```

Restul codului de testare este suficient de clar pentru a nu mai insista aici. Urmariti comentariile din cod.
O diagrama de secvente aferenta primei parti a acestui cod de test este disponibila in figura 15.

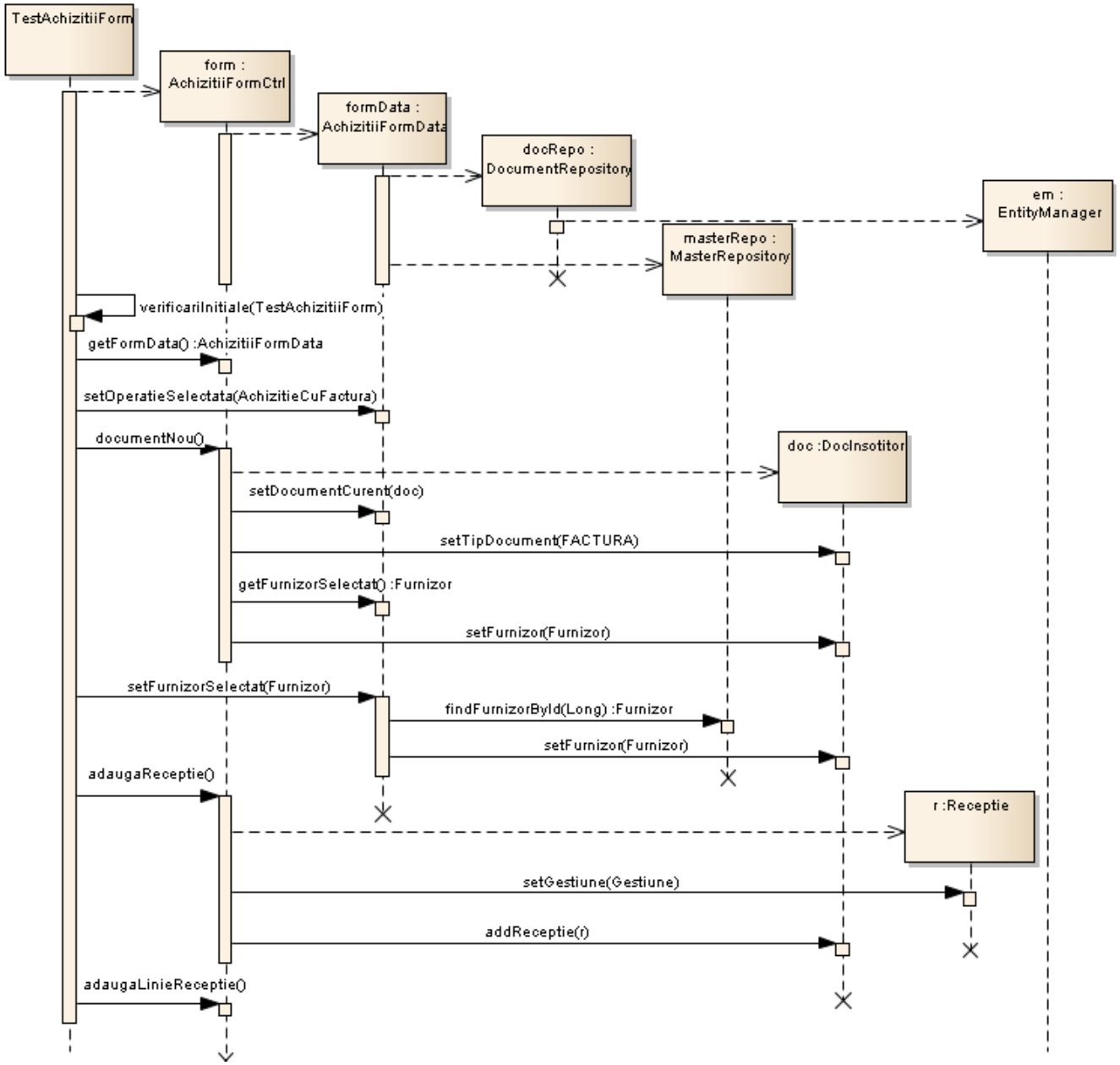


Figura 15 Diagrama de secvente pentru prima parte a scenariului de adaugare